

DOCUMENTO DE ANÁLISIS DE REQUISITOS

Aplicación: <Saimazoom>

FECHA: <26/03/2025>

1. Introducción

1.1 PROPÓSITO DEL SISTEMA

La finalidad de este sistema es proporcionar una solución software distribuida que permita gestionar de manera eficiente la cadena de procesamiento de pedidos en el entorno logístico de Saimazoom. Este sistema permite coordinar a todos los actores involucrados: clientes, controlador central, robots y repartidores, a través de un modelo basado en colas de mensajes. El objetivo principal es asegurar que los pedidos de los clientes se gestionen, tramiten, transporten y entreguen correctamente, garantizando la trazabilidad y eficiencia del proceso.

1.2 ÁMBITO DEL SISTEMA

El sistema está diseñado para ser utilizado por múltiples entidades que colaboran en la logística de pedidos. Está compuesto por varios actores bien diferenciados:

- **Cliente registrado**, que tiene la capacidad de registrarse en la aplicación, autenticarse, realizar pedidos de productos, consultar su estado, cancelarlos si es necesario y recibir notificaciones sobre su evolución.
- **Controlador central**, que actúa como cerebro del sistema y es responsable de recibir los pedidos, asignar tareas a los robots y repartidores, gestionar los estados del pedido, coordinar la comunicación entre todos los actores y asegurar la persistencia de la información.
- **Robots**, que son asignados por el controlador para localizar los productos en el almacén y transportarlos hasta la cinta transportadora. En caso de no encontrar los productos, informan del fallo.
- **Repartidores**, que recogen los productos desde la cinta y los entregan en el domicilio del cliente, confirmando su recepción.

Cada uno de estos actores se comunica a través de colas de mensajes independientes mediante RabbitMQ, asegurando un flujo de información desacoplado, fiable y escalable.

1.3 OBJETIVOS Y CRITERIOS DE ÉXITO DEL PROYECTO

Los principales objetivos del sistema, así como los criterios que se utilizarán para determinar su éxito, son los siguientes:

- Cumplir los requisitos funcionales y no funcionales establecidos para cada actor del sistema.
- Permitir una administración eficiente de pedidos y estados mediante comunicación distribuida.
- Garantizar una interfaz clara y sencilla para los clientes, con una presentación de la información consistente y sin necesidad de conocimientos técnicos avanzados.

- Asegurar el correcto funcionamiento del sistema bajo distintas condiciones, reduciendo al mínimo los errores mediante una lógica de control robusta.
- Proporcionar una experiencia fluida al usuario, con tiempos de respuesta rápidos y notificaciones en tiempo real.
- Facilitar el mantenimiento y la ampliación del sistema en el futuro, mediante una arquitectura modular y bien definida.

1.4 DEFINICIONES, ACRÓNIMOS Y ABREVIATURAS

Lista con términos técnicos y abreviaturas que se usarán en el resto del documento.

2. Descripción del sistema

2.1 REQUISITOS FUNCIONALES

Tipos de usuario y descripción de las funciones que puede realizar cada uno de ellos.

2.1.1 Controlador Central

- 2.1.1.1.** Hacer login en el sistema como entidad de control.
- 2.1.1.2.** Recibir y registrar pedidos enviados por clientes a través de RabbitMQ.
- 2.1.1.3.** Asignar un robot disponible para procesar un pedido.
- 2.1.1.4.** Asignar un repartidor una vez que los productos han sido colocados en la cinta.
- 2.1.1.5.** Actualizar el estado del pedido tras cada fase: "pendiente", "en reparto", "completado", "no encontrado", "cancelado", etc.
- 2.1.1.6.** Cancelar un pedido si el cliente lo solicita antes de ser recogido por el repartidor.
- 2.1.1.7.** Notificar al cliente el estado de su pedido mediante colas de mensajes.
- 2.1.1.8.** Registrar clientes, pedidos y mantener el estado de los mismos.
- 2.1.1.9.** Comunicar fallos a los actores correspondientes (robots, repartidores, cliente).
- 2.1.1.10.** Coordinar y monitorear el funcionamiento del sistema global.

2.1.2 Cliente Registrado

- 2.1.2.1.** Registrarse en el sistema proporcionando identificador y contraseña.
- 2.1.2.2.** Iniciar sesión en la aplicación mediante su identificador y contraseña.
- 2.1.2.3.** Realizar un pedido indicando número de productos.
- 2.1.2.4.** Consultar el estado actual de uno o varios pedidos realizados.
- 2.1.2.5.** Cancelar un pedido antes de que sea entregado.
- 2.1.2.6.** Recibir notificaciones sobre la entrega, errores o cancelaciones de pedidos.
- 2.1.2.7.** Confirmar recepción del pedido.
- 2.1.2.8.** Visualizar su historial de pedidos.

2.1.3 Robot

- 2.1.3.1.** Recibir órdenes del controlador para buscar productos en el almacén.
- 2.1.3.2.** Buscar el producto correspondiente en el almacén.
- 2.1.3.3.** Si encuentra el producto, colocarlo en la cinta transportadora.
- 2.1.3.4.** Si no lo encuentra, notificar el fallo al controlador.
- 2.1.3.5.** Recibir y ejecutar notificaciones de cancelación de pedido si ya está en curso.
- 2.1.3.6.** Volver al almacén si el pedido ha sido cancelado.

2.1.4 Repartidor

- 2.1.4.1.** Recibir órdenes del controlador para recoger productos desde la cinta.
- 2.1.4.2.** Transportar el pedido al domicilio del cliente.
- 2.1.4.3.** Confirmar la entrega al controlador.
- 2.1.4.4.** Notificar fallos en la entrega si se presentan incidencias.

2.2 REQUISITOS NO FUNCIONALES

- **Portabilidad:** Debe funcionar en diferentes entornos y sistemas operativos.
- **Interfaz simple e intuitiva:** Para usuarios no expertos (clientes).
- **Fiabilidad en la mensajería:** Comunicación robusta vía RabbitMQ.
- **Escalabilidad:** Capaz de soportar múltiples clientes, pedidos concurrentes y gestión de robots y repartidores en paralelo.
- **Tolerancia a fallos:** Capacidad para gestionar errores de hardware o software sin colapsar el sistema.
- **Consistencia del sistema:** Estados de los pedidos sincronizados en todo momento.
- **Tiempo de respuesta bajo:** Idealmente <1 segundo entre envío de petición y respuesta visible.
- **Registro y trazabilidad:** Todos los eventos relevantes deben quedar registrados para revisión o auditoría.
- **Seguridad:** Autenticación y autorización segura para cada actor.

3.2 DESCRIPCIÓN DE LOS CASOS DE USO

Detalle de 3 casos de uso del sistema. Utiliza la siguiente plantilla:

3.2.1. CASO DE USO <Pedido completado hasta el final>

Actor Primario:

Cliente

Interesados y Objetivos:

- Cliente Registrado: Completar un pedido exitosamente desde su creación hasta la entrega.

Precondiciones:

El cliente debe estar registrado y haber iniciado sesión.

Postcondiciones:

El pedido se entrega al cliente con éxito.

Garantía de éxito (Postcondiciones):

El pedido se entregará correctamente siempre que el robot encuentre el producto y el repartidor complete la entrega en alguno de sus tres intentos.

Escenario Principal de Éxito:

1. El cliente envía un mensaje ORDER client_id productos.
2. El controlador genera un nuevo order_id y guarda el pedido en estado PENDING.
3. El controlador reenvía el mensaje MOVE order_id a la cola de robots.
4. El robot responde con MOVE_SUCCESS order_id.
5. El controlador cambia el estado del pedido a ON_CONVEYOR y envía DELIVERY order_id a la cola de repartidores.
6. El repartidor intenta la entrega y responde con DELIVERY_SUCCESS order_id.
7. El controlador actualiza el estado a DELIVERED y notifica al cliente.

Extensiones (Flujos alternativos):

Si el robot falla, se detiene el proceso y se notifica al cliente (ver caso 2).

Si el reparto falla tras 3 intentos, el estado pasa a FAILED.

Requisitos Especiales:

Respuesta y actualización rápidas (<0.1 seg) al procesar la compra.

Frecuencia de ocurrencia:

Orden de cientos de miles, se estima que esta operación se realice cientos de miles de veces a lo largo de la vida útil del software.

Temas abiertos:

<Cuestiones que están abiertas y se plantean para resolución futura, o para considerarse en futuras versiones de la aplicación>

Considerar que se pidan más datos al cliente para tramitar la compra.

3.2.2. CASO DE USO: <Pedido en el que el robot no encuentra el producto>**Actor Primario:**

Cliente registrado

Interesados y Objetivos:

- **Cliente registrado:** Quiere recibir su pedido o, en su defecto, ser informado para tomar una decisión.
- **Controlador central:** Desea gestionar correctamente pedidos con incidencias.
- **Robot:** Intenta localizar los productos en el almacén.
- **Repartidor:** (No participa si el producto no se encuentra).

Precondiciones:

- El cliente debe estar registrado y autenticado.
- El pedido debe haber sido creado y enviado al controlador.
- El sistema debe estar operativo y con robots disponibles.

Garantía de éxito (Postcondiciones):

- El estado del pedido se actualiza a "producto(s) no encontrado(s)".
- El cliente recibe una notificación y puede actuar (modificar/cancelar).
- No se realiza entrega si no se encuentra el producto.

Escenario Principal de Éxito:

- 1) El cliente realiza un ORDER como en el caso anterior.
- 2) El controlador crea el pedido en estado PENDING.
- 3) Envía MOVE order_id a la cola de robots.
- 4) El robot responde con MOVE_FAILURE order_id.
- 5) El controlador actualiza el estado a NOT_FOUND y responde al cliente con FAILURE Producto no encontrado.

Extensiones (Flujos alternativos):

Ninguna. El flujo se detiene tras el fallo del robot.

Requisitos Especiales:

- Detección inmediata de productos no disponibles.
- Notificación al cliente rápida (<1s idealmente).
- Opcional: sugerencia de productos similares por parte del sistema.

Frecuencia de ocurrencia:

Moderada. Puede depender de la gestión del inventario. Se espera que ocurra ocasionalmente.

Temas abiertos:

- ¿El sistema debe ofrecer productos alternativos automáticamente?
- ¿Debe permitirse la reserva del resto de productos mientras el cliente decide?

3.2.3. CASO DE USO: <Pedido que se cancela antes de empezar el reparto>

Actor Primario:

Cliente registrado

Interesados y Objetivos:

- **Cliente registrado:** Desea cancelar un pedido que ya no quiere recibir.
- **Controlador central:** Debe permitir la cancelación del pedido antes del envío.
- **Robot:** Debe ser notificado para no entregar el producto a la cinta transportadora.
- **Repartidor:** (No interviene si el pedido se cancela antes del reparto).

Precondiciones:

- El cliente debe estar registrado y autenticado.
- El pedido debe estar en estado PENDING o IN_STORE.
- El robot puede estar ya asignado o en proceso de buscar los productos.

Garantía de éxito (Postcondiciones):

- El pedido queda registrado con estado "cancelado".
- El cliente recibe una notificación confirmando la cancelación.
- El robot es notificado y regresa al almacén sin entregar los productos.

Escenario Principal de Éxito:

- 1) El cliente realiza un ORDER como en los casos anteriores.
- 2) El controlador guarda el pedido.
- 3) Antes de que el controlador reciba MOVE_SUCCESS, el cliente envía CANCEL order_id.
- 4) El controlador verifica que el estado es cancelable (PENDING o IN_STORE).
- 5) Cambia el estado del pedido a CANCELED.
- 6) Notifica al cliente con CANCEL_SUCCESS order_id.

Extensiones (Flujos alternativos):

- Si el pedido ya está en reparto o entregado, se responde con FAILURE Pedido ya en reparto o entregado.

Requisitos Especiales:

- Cancelación efectiva solo si el pedido aún no ha sido recogido por el repartidor.
- Comunicación inmediata con el robot para evitar la entrega del producto.
- Registro automático del evento de cancelación para trazabilidad.

Frecuencia de ocurrencia:

Moderada. Ocurre ocasionalmente cuando los usuarios cambian de opinión tras realizar un pedido.

Temas abiertos:

- ¿El sistema debe permitir cancelaciones en pedidos parciales (si algunos productos ya están listos y otros no)?
- ¿Se debe aplicar alguna penalización o condición adicional para cancelaciones frecuentes?

Diagrama de clases

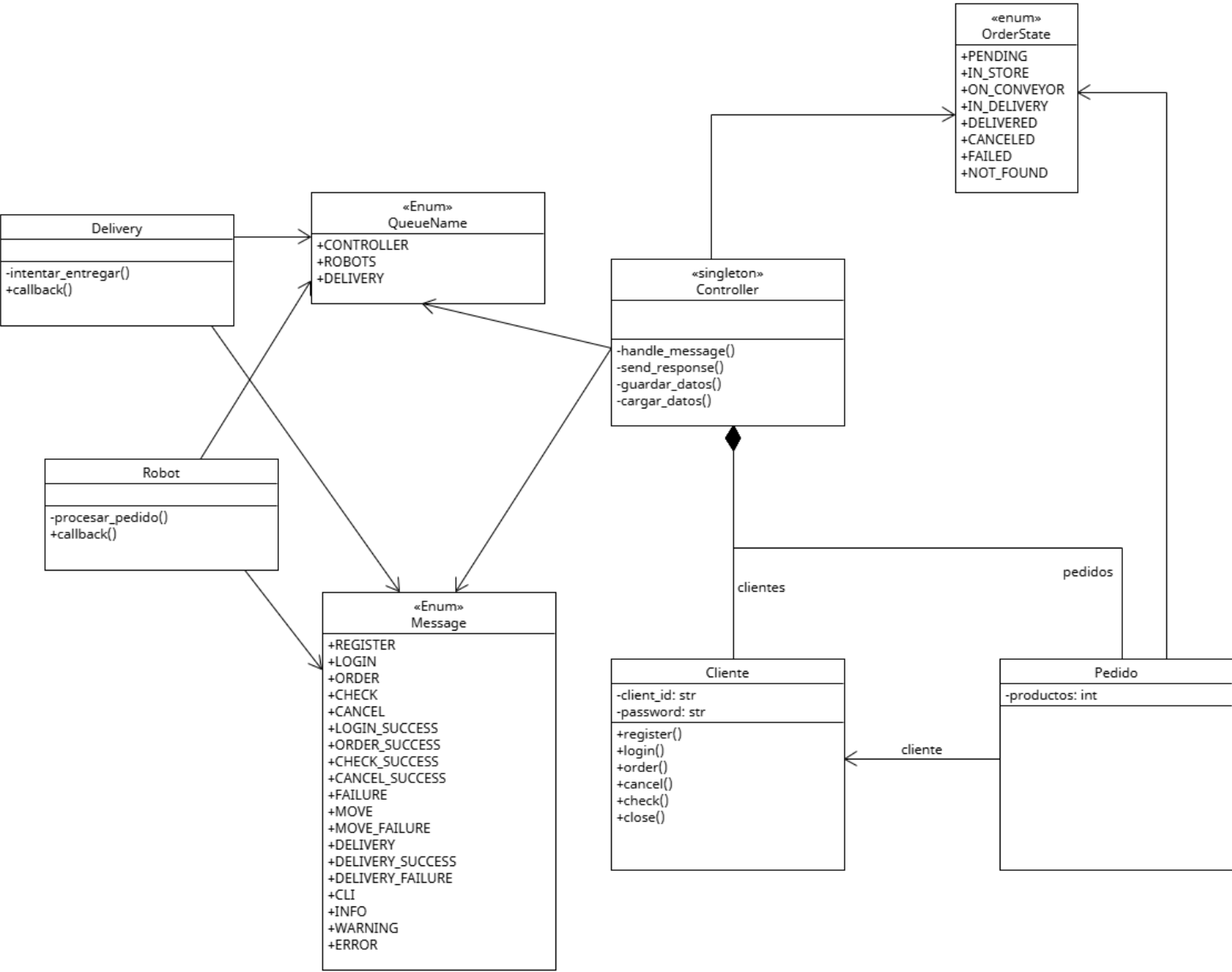


Diagrama Estados pedidos

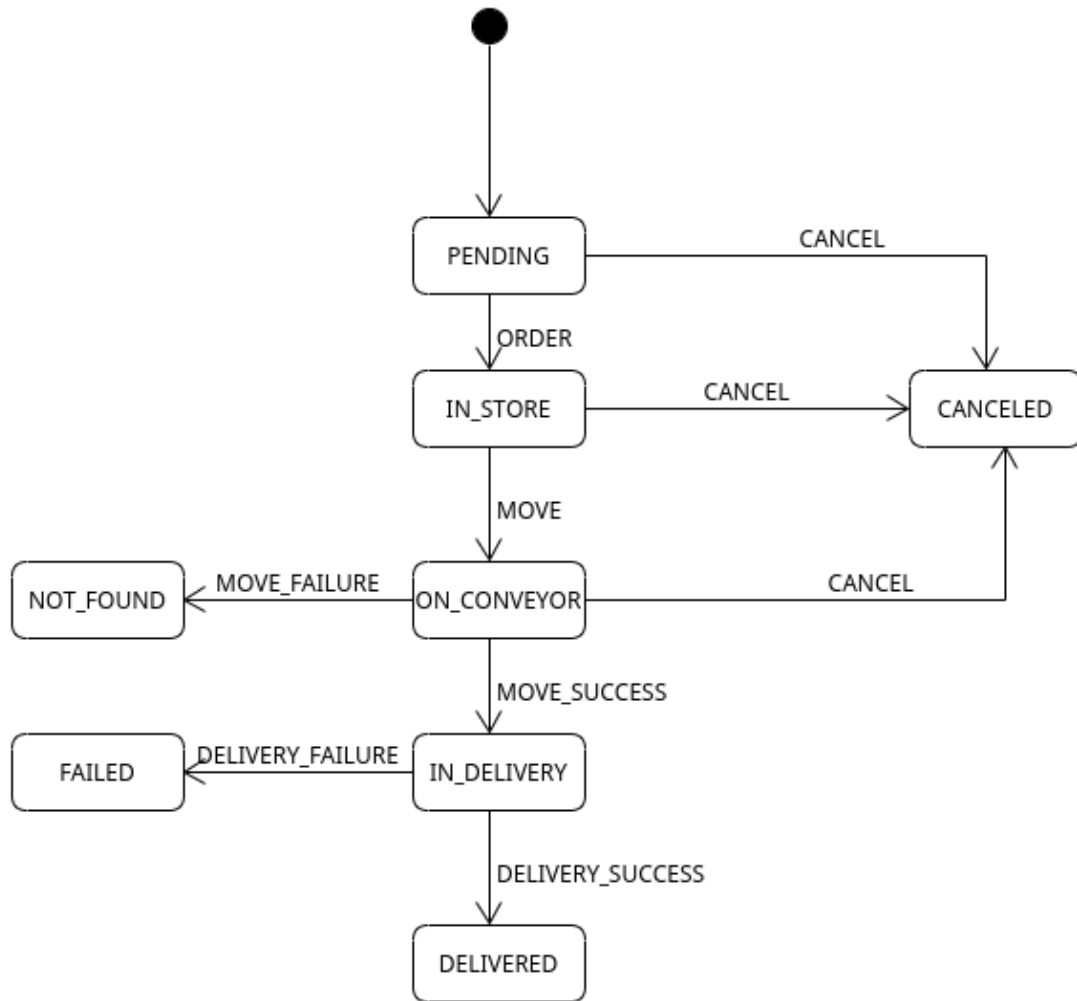


Tabla de mensajes

Mensaje	De	A	Parámetros	Acción	Descripción
REGISTER	Cliente	Controlador	client_id, password	Registrar Cliente	Alta de un nuevo cliente
LOGIN	Cliente	Controlador	client_id, password	Validar login	Autenticación del cliente
ORDER	Cliente	Controlador	client_id, n_products	Crear pedido	Solicitud de pedido de productos
CHECK	Cliente	Controlador	order_id	Consultar estado	Consulta el estado actual de un pedido
CANCEL	Cliente	Controlador	order_id	Cancelar pedido	Cancelación del pedido (si no ha sido entregado aún)
ROBOT	Controlador	Robot	order_id	Buscar producto	Instrucción para que el robot busque productos del pedido
ROBOT_SUCCESS	Robot	Controlador	order_id	Confirmar búsqueda	El robot encontró el producto
ROBOT_FAILURE	Robot	Controlador	order_id	Notificar fallo	El robot no encontró el producto
DELIVERY	Controlador	Repartidor	order_id	Iniciar entrega	Solicitud para que el repartidor entregue el pedido
DELIVERY_SUCCESS	Repartidor	Controlador	order_id	Confirmar entrega	El pedido se entregó correctamente
DELIVERY_FAILURE	Repartidor	Controlador	order_id	Notificar fallo	Fallo en la entrega del pedido
LOGIN_SUCCESS	Controlador	Cliente	client_id	Confirmar login	Notificación de éxito al iniciar sesión
ORDER_SUCCESS	Controlador	Cliente	order_id	Confirmar creación de pedido	Notificación de que el pedido se creó correctamente
CHECK_SUCCESS	Controlador	Cliente	order_id, state	Mostrar estado	Respuesta con el estado actual del pedido
CANCEL_SUCCESS	Controlador	Cliente	order_id	Confirmar cancelación	Notificación de cancelación exitosa del pedido
FAILURE	Controlador	Cliente	-	Mostrar error	Mensaje de error en alguna operación
CLI	*	Cliente	client_id, texto_msg	Imprimir en terminal	Mensaje informativo dirigido al usuario
I/W/E	*	Logger	texto_msg	Registrar en log	Mensaje informativo (I), de aviso (W) o de error (E) en el log

Sintaxis Mensajes

REGISTER = "REGISTER" # REGISTER <client_id> <password>

LOGIN = "LOGIN" # LOGIN <client_id> <password>

ORDER = "ORDER" # ORDER <client_id> <n_products>

CHECK = "CHECK" # CHECK <order_id>

CANCEL = "CANCEL" # CANCEL <order_id>

LOGIN_SUCCESS = "LOGIN_SUCCESS" # LOGIN_SUCCESS
<client_id>

ORDER_SUCCESS = "ORDER_SUCCESS" # ORDER_SUCCESS
<order_id>

CHECK_SUCCESS = "CHECK_SUCCESS" # CHECK_SUCCESS
<order_id> <state>

CANCEL_SUCCESS = "CANCEL_SUCCESS" # CANCEL_SUCCESS
<order_id>

FAILURE = "FAILURE" # FAILURE

ROBOT = "MOVE" # MOVE <order_id>

ROBOT_SUCCESS = "MOVE_SUCCESS" # MOVE_SUCCESS
<order_id>

ROBOT_FAILURE = "MOVE_FAILURE" # MOVE_FAILURE <order_id>

DELIVERY = "DELIVERY" # DELIVERY <order_id>

DELIVERY_SUCCESS = "DELIVERY_SUCCESS" #

DELIVERY_SUCCESS <order_id>

DELIVERY_FAILURE = "DELIVERY_FAILURE" # DELIVERY_FAILURE
<order_id>

CLI = "CLI" # CLI <client_id> <text_message>

INFO = "I" # I <text_message>

WARNING = "W" # W <text_message>

ERROR = "E" # E <text_message>

Opcional (futuro)

CONFIRM_RECEIPT = "CONFIRM_RECEIPT" # CONFIRM_RECEIPT
<order_id>